

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Belagavi, Karnataka-590018



A MINI PROJECT SYNOPSIS
ON
HIGH-SPEED DATA TRANSFER SYSTEM

Submitted in partial fulfillment of the requirements for the degree of
BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE & BUSINESS SYSTEM

Submitted by

4CB24CB007 Aravind
4CB24CB050 Swananda Shridhar Hegde
4CB24CB026 Manish S A

Under the guidance of

Yojana Kiran Kumar

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE & BUSINESS SYSTEM
CANARA ENGINEERING COLLEGE
SUDHINDRA NAGARA, BENJANAPADAVU, BANTWAL – 574219
2026–2027

ABSTRACT

Modern computer applications frequently require users to access, manage, and share large amounts of digital data across multiple systems. Applications such as file-sharing platforms, media servers, collaborative systems, and distributed storage services require a server capable of managing multiple users, handling requests, maintaining data, and efficiently communicating with connected clients. As the amount of data and number of users increase, inefficient application design and unnecessary movement of data between memory, storage, and network components can affect system responsiveness and resource utilization.

This project proposes a Linux-based client-server application for managing and sharing data between participating systems. The primary focus of the project is the development of the server and client applications, including connection management, client sessions, resource management, file handling, transfer requests, and communication between multiple connected clients. The server acts as the central component responsible for maintaining available data resources and coordinating requests from clients. A client interacts via an intuitive frontend UI application linked to an integrated client backend engine with built-in eBPF drivers to discover or request available resources, initiate an operation, and receive requested data. The system is designed to support both Host-to-Host communication and Host-to-Many data distribution, allowing data hosted by one system to be accessed by multiple participating clients.

The application is designed around modular components so that connection handling, request processing, data management, transfer control, and internal buffering can be developed and tested independently. The server maintains information about active client sessions and ongoing operations while coordinating the movement of data between storage, memory, and network interfaces. Appropriate sequencing, buffering, validation, and resource management mechanisms are used to maintain the correctness of operations when multiple clients are active simultaneously.

To improve the efficiency of the application, the project also investigates techniques for reducing unnecessary data movement inside the system. Reusable buffers and shared memory can be used by different components instead of repeatedly allocating and copying large payloads. Ring buffers can provide controlled communication between processing components, while descriptor-based buffer management allows metadata such as buffer identifiers, offsets, lengths, sequence information, and ownership state to be exchanged independently of the complete payload.

The project further investigates Linux technologies such as XDP, eBPF, and AF_XDP as supporting mechanisms for improving selected parts of the application's data path for efficient network and storage data handling on the client side.

PROBLEM STATEMENT

Applications that manage and share data between multiple systems require a server capable of handling client connections, processing requests, managing resources, and coordinating concurrent operations. As the amount of data and number of clients increase, inefficient buffering and repeated data movement can increase processing overhead and affect application performance.

The project addresses the development of a modular client-server application that can manage multiple clients and data resources efficiently while incorporating suitable Linux-based mechanisms to reduce unnecessary data movement and improve the performance of selected application components. Standard socket implementations incur significant kernel-to-user-space copying overhead that degrades throughput under high load, necessitating optimized buffering strategies and asynchronous I/O architectures.

OBJECTIVES

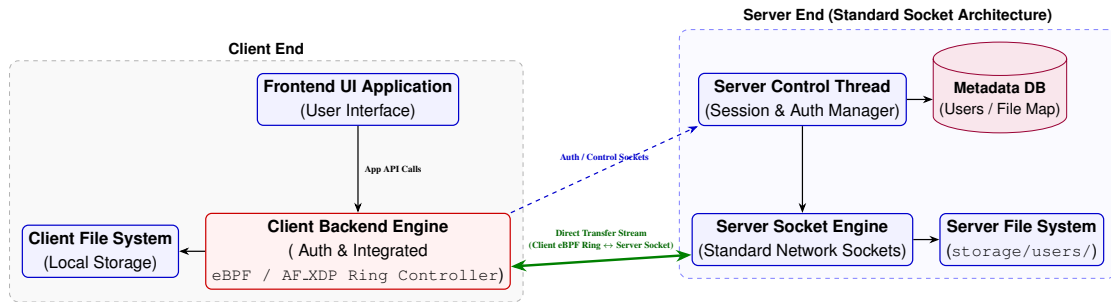
1. **Architectural & Application Layer Design:** Design and develop a modular client-server framework incorporating a Frontend UI linked to a high-performance Client Backend Engine. The system will handle active session tracking, authentication handshakes, multi-threaded request scheduling, and file discovery protocols across both Host-to-Host and Host-to-Many data distribution topologies.
2. **Kernel-Bypass & Optimized Data Path Integration:** Construct an optimized, low-latency network data path by embedding `eBPF` and `AF_XDP` (Express Data Path) driver hooks into the client subsystem. This objective focuses on bypassing the conventional TCP/IP kernel stack through lockless ring buffers (`Fill`, `Completion`, `Rx`, and `Tx`), enabling raw packet descriptors to be passed directly into user-space shared memory pools with minimal processing overhead.
3. **Asynchronous Storage Management & Performance Benchmarking:** Implement an asynchronous non-blocking disk I/O interface for high-efficiency storage access, and conduct comprehensive comparative evaluations. The goal is to rigorously measure throughput (Gbps), packet processing latency, CPU utilization, and system overhead against standard synchronous POSIX storage and traditional network socket architectures under high client concurrency.

METHODOLOGY

The proposed system adopts a dual-plane architecture separating user interface controls and administrative signaling from low-level, high-throughput client-side eBPF/AF_XDP kernel-bypass acceleration. The methodology encompasses component design, memory management, data-path execution flows, and storage integration.

System Architecture & Component Breakdown

The overall architecture is structured into two primary operational domains—the **Client End** and the **Server End**—connected via control sockets and accelerated direct data streams:



Operational Workflow & Data Path Execution

The complete system workflow operates across four sequential execution phases:

- Control Plane Handshake & Session Auth:** The user initiates an operation via the Frontend UI. The Client Backend Engine transmits authentication tokens over standard TCP control sockets to the Server Control Thread, which verifies credentials against the Metadata Database.
- AF_XDP UMEM Initialization & Ring Buffer Setup:** Upon session confirmation, the client allocates an aligned user-memory area (UMEM) in user space. Lockless ring buffers (Fill, Completion, Rx, and Tx rings) are registered with the Linux network interface card (NIC) driver using `libxdp` and `libbpf`.
- Asynchronous Storage Subsystem Integration:** Disk read/write operations utilize an asynchronous, non-blocking I/O pipeline. This eliminates synchronous file I/O blocking during disk-to-memory transfers on both endpoints.
- Direct Kernel-Bypass Data Stream:** Incoming network frames are intercepted at the NIC level by eBPF/XDP hooks and deposited directly into user-space UMEM buffers. By passing frame descriptors rather than copying packet payloads across kernel boundaries, high-throughput transfers are achieved with minimal CPU overhead.

REFERENCES

1. Z. Cao et al., “ChainIO: Bridging Disk and Network Domains with eBPF,” Proceedings of the 3rd Workshop on eBPF and Kernel Extensions, 2025.
<https://dl.acm.org/doi/10.1145/3672197.3673434>
2. A. B. Narappa et al., “Z-Stack: A High-Performance DPDK-Based Protocol Stack,” IEEE LANMAN, 2024.
<https://ieeexplore.ieee.org/document/10599021>
3. J. Axboe, “Efficient I/O with io_uring,” Linux Kernel Documentation & Official liburing Repository.
<https://github.com/axboe/liburing>
4. eBPF and XDP Documentation, “XDP Helper Library ”
<https://github.com/xdp-project/xdp-tools>